

abVisualRTL

A Hands-On Tutorial

Draw · Define · Generate

SystemVerilog & VHDL

rev. July 2026

abVisualRTL turns a state diagram into synthesizable hardware. You draw states and the transitions between them; the tool writes clean SystemVerilog or VHDL, live, as you work. This guide takes you from an empty canvas to an exported, validated FSM, in the order you'll actually build one. Work through it once start to finish; after that, the [shortcut reference](#) on the last page is all you'll need at your elbow.

CONTENTS

- | | |
|---|---|
| 1 The workspace | 8 Module, clock & ports |
| 2 Grid, snap, zoom & pan | 9 Output defaults & latches |
| 3 Your first state | 10 The live preview |
| 4 State names & Moore actions | 11 Validation |
| 5 Transitions & the router | 12 Saving & exporting |
| 6 Transition logic & priority | 13 Worked example: a detector |
| 7 Mealy actions | 14 Shortcut reference |

1 The workspace

When abVisualRTL opens you're looking at four regions. Learn their names now; the rest of the guide uses them constantly.

- **Toolbar** (left) — the three drawing tools: [Select](#), [State](#), and [Line](#). Select moves and edits; State places new states; Line draws transitions between them.
- **Canvas** (center) — the drawing surface, with a dotted alignment grid. States are circles; transitions are smooth B-spline curves. Everything snaps to the grid.

- **Live preview** (right) — a split-screen pane showing the generated HDL. It regenerates every time you touch the diagram.
- **Menu bar** (top) — **File** (new / open / save / export), **Edit** (undo / redo), and **Settings** for the module, clock, reset and ports.

MENTAL MODEL

A diagram is just **states** and **transitions** over a fixed clock. Every state is one value of a state register; every transition is a condition checked on each clock edge. Keep that in mind and the generated code will always read the way you expect.

2 Grid, snap, zoom & pan

A tidy diagram is a snapped diagram. The dotted grid keeps states aligned and curves predictable, and it's drawn only across the visible region so large diagrams stay responsive.

Zoom works two ways:

- **Zoom controls** — step in and out; the diagram redraws crisply at each level with labels resized to match.
- **Pan** — use the scrollbars at the canvas edges to move around a diagram larger than the window.

GOOD TO KNOW

Zoom is purely a *view*. It never changes what's saved, generated, or exported — an FSM viewed at 200% produces byte-for-byte the same HDL as one viewed at 100%. Zoom freely; you can't hurt anything.

3 Your first state

Every FSM is a handful of states. Let's place one.

1. Click the **State** tool in the toolbar.

2. Click anywhere on the canvas. A circular state appears, snapped to the grid, with an auto-generated name.
3. Switch to the **Select** tool. Click the state to select it (a highlight and move cursor appear).
4. Drag it to reposition. Any transitions attached to it follow automatically.

PICK A RESET STATE

Every FSM needs exactly one **reset state** — where the machine lands when reset is asserted. You'll mark it in the next step. abVisualRTL enforces the "exactly one" rule for you, so you never generate a machine with no defined start.

4 State names & Moore actions

Double-click a state to open its editor. Here you set three things:

- **Name** — the enum label for this state, e.g. `IDLE`. It must be a valid identifier and unique (see section 11).
- **Is Reset State** — tick this on your starting state. Setting it here clears the flag on any other state, so there is always exactly one.
- **Moore actions** — outputs driven whenever the machine is *in* this state. One per line:

```
busy = 0           // SystemVerilog
busy <= '0'        -- VHDL
```

MOORE VS. MEALY

A **Moore** action lives inside a state and depends only on *which* state you're in. A **Mealy** action (section 7) lives on a transition and fires only when that transition is taken. Use Moore for steady per-state outputs; use Mealy for one-cycle pulses tied to an event.

5 Transitions & the router

Transitions are the arrows that move the machine between states. With the **Line** tool selected:

- **State to state** — press on the source state, drag to the target, release. A smooth B-spline curve connects them, entering and leaving each circle cleanly.
- **Self-loop** — drag a state back onto itself to create a loop, for "stay here while the condition holds."
- **Reshape** — with the **Select** tool, drag a transition's midpoint handle to bend the curve; drag its label to move the text out of the way.

THE ONE RULE

A transition is **owned by the states it connects**. Move either state and the curve rebuilds itself to stay attached, always meeting the circle at a clean angle. You draw intent; the router keeps it tidy as things move.

6 Transition logic & priority

Double-click a transition to open its editor. You set:

- **Condition** — the boolean that fires this transition, e.g. `start == 1`. Leave it blank (or use `1`) for an unconditional default edge.
- **Priority** — when several transitions leave one state, lower numbers are tested first. The first condition that holds wins; a default edge should carry the highest number so it's the fallback.

NO TIES ALLOWED

Two transitions leaving the same state may not share a priority — the order would be undefined. abVisualRTL blocks code generation and names the offending pair so you can renumber. Give every outgoing edge a distinct priority.

7 Mealy actions

A transition can also carry an **action** — an output driven for the cycle in which that transition is taken. In the transition editor, add it below the condition:

```
condition: data_valid == 1
action:    capture = 1
```

On the diagram, a transition's label reads `[condition] / action` — the bracketed guard, then the Mealy action after the slash. This is the standard state-diagram notation, so your drawing is self-documenting.

8 Module, clock & ports

Open [Settings](#) to define the module's interface — everything outside the state logic itself.

- **Module name, clock, and reset** — the identifiers for the generated module and its two control signals.
- **Ports** — add each I/O signal with a **direction** (input / output / inout), a **bit width**, and, for outputs, a **default value** (section 9). Names are checked as you add them, and can't collide with the clock or reset.

ONE NAMESPACE

In hardware, state names, port names, the clock and the reset all share one namespace — especially in VHDL, which folds case. abVisualRTL treats `Idle` and `IDLE` as the same name and warns before they can clash.

9 Output defaults & latch prevention

Each output port carries a **default value**, set in the ports list. abVisualRTL emits it at the top of the combinational block, so every output holds a defined value even in states and transitions where you don't assign it.

WHY THIS MATTERS

An output assigned in only *some* states infers an unwanted **latch** in synthesis — a classic, hard-to-spot HDL bug. Default values make the logic fully combinational and safe. Set a sensible default (usually `0`) for every output and you'll never chase a latch warning.

10 The live preview

The right-hand pane shows the generated HDL and refreshes on every change — draw a transition, and the corresponding `case` branch appears instantly. Switch the target language between **SystemVerilog** and **VHDL** and the whole pane regenerates. The state register's bit width is sized automatically to the number of states, so it's always exactly wide enough.

11 Validation

Before it writes a line of HDL, abVisualRTL checks your design and reports anything that would produce broken or ambiguous code:

- **Legal identifiers** — state, port, module, clock and reset names must be valid identifiers and must not be reserved words in *either* SystemVerilog or VHDL.
- **Unique names** — no duplicate states or ports (case-insensitive), and no collision between a state name and a signal name.
- **Distinct priorities** — no two transitions leaving one state may share a priority.

Problems are listed so you can fix them before synthesis — the generator never emits code it knows won't compile.

12 Saving & exporting

Save writes your whole project — diagram, settings and ports — to one JSON file you can reopen later. That's your working format, and its structure is validated on load.

To hand off or document, use **File > Export**:

- **SystemVerilog / VHDL** — the generated HDL, ready for your synthesis toolchain.
- **Diagram as PNG / PDF** — the full diagram rendered off-screen (not just the visible area), for documentation and reviews.

CLEAN EXPORTS

The diagram export renders the entire FSM at high resolution regardless of zoom or scroll position, with the reset state marked by a double circle. What you drew is exactly what prints.

13 Worked example: a "1011" detector

Let's tie it together by building a serial sequence detector that pulses `found` whenever the bit stream `1011` arrives. Follow along; every step uses something from the sections above.

1. **Set up the interface.** In **Settings**, add input `din` (1 bit) and output `found` (1 bit, default `0`).
2. **Place the states.** With the State tool, make five: `S0`, `S1`, `S10`, `S101`, `S1011` — each name tracks the prefix matched so far.
3. **Mark the start.** Double-click `S0` and tick **Is Reset State**.
4. **Wire the "match" path.** With the Line tool, draw the forward edges — `S0→S1` on `din==1`, `S1→S10` on `din==0`, `S10→S101` on `din==1`, `S101→S1011` on `din==1`.
5. **Pulse the output.** On the edge landing in `S1011`, add the Mealy action `found = 1` so it fires for exactly one cycle.
6. **Add the "miss" paths.** Draw the fallback edges (e.g. `S1→S1` on another `din==1`, wrong bits back toward `S0`) and give each a distinct **priority** so the match path is tested first.
7. **Verify.** Watch the live preview fill in; if anything's off, the validator names it. Fix names or priorities as flagged.
8. **Deliver.** Save the project, then **File > Export** the SystemVerilog or VHDL — and a PDF of the diagram for the design doc.

That's the whole loop — interface, states, reset, transitions, actions, priority, verify, export. Every larger FSM is just more of the same.

14 Shortcut reference

FILE

TOOLS

New	Ctrl+N
-----	--------

Open	Ctrl+O
------	--------

Save	Ctrl+S
------	--------

Export diagram	File › Export
----------------	---------------

EDIT

Undo / Redo	Ctrl+Z / Ctrl+Y
-------------	-----------------

Delete selected	Delete
-----------------	--------

Edit state / transition	Double-click
-------------------------	--------------

Select / move	Select tool
---------------	-------------

Add state	State tool + click
-----------	--------------------

Add transition	Line tool + drag
----------------	------------------

Self-loop	drag onto self
-----------	----------------

DESIGN

Module & ports	Settings
----------------	----------

Language	SV / VHDL
----------	-----------

Export HDL	File › Save HDL
------------	-----------------